

Marco 1 - Paralelização do Conjunto de Mandelbrot

Análise do algoritmo sequencial, perfilamento, baseline e plano de paralelização

Disciplina	Programação Paralela (CCO085) - IESB 2026/2
Professor	Rodrigo Gonçalves Pinto
Problema	No 3 - Conjunto de Mandelbrot
Integrantes	Yuri Victor de Oliveira e Silva - Gabriel Vieira
Repositório	github.com/yvos77/projeto_progpl
Máquina	Intel Core i5-11400F (6 núcleos físicos, 12 threads lógicas)
Ambiente	Ubuntu sobre WSL2 (Windows), VS Code Remote-WSL; g++ -O2, OpenMPI

1. O que o algoritmo faz

O programa varre uma imagem de $W \times H$ pixels. Cada pixel (x, y) é mapeado para um ponto $c = c_r + i.c_i$ do plano complexo, com c_r variando linearmente em $[-2,5; 1,0]$ e c_i em $[-1,25; 1,25]$. Para esse c , a sequência $z_0 = 0$, $z_{n+1} = z_n^2 + c$ é iterada até que $|z_n|^2 > 4$, caso em que o ponto escapou e não pertence ao conjunto, ou até atingir o teto $MAXIT = 1000$, caso em que o ponto é tratado como pertencente ao conjunto. O valor gravado em $hist[y.W + x]$ é o número de iterações até o escape. Ao final o programa soma todas as posições do vetor e imprime esse somatório como checksum, que é o gabarito de correteude.

2. Complexidade assintótica

O custo do pixel (x, y) é $it(x, y)$, limitado por $MAXIT$. O custo total é:

$$T(W, H) = \text{SOMA}_y \text{ SOMA}_x it(x, y) = O(W \cdot H \cdot MAXIT)$$

O limite superior só é justo para os pixels internos ao conjunto, que percorrem as 1000 iterações completas. Para os pixels externos, it é pequeno, tipicamente algumas dezenas. O custo médio efetivo por pixel é portanto muito menor que $MAXIT$, mas a variância entre pixels é enorme, e é essa variância que domina o comportamento paralelo do problema. O consumo de memória é $O(W.H)$, com 8 bytes por pixel.

3. Perfilamento

A instrumentação está em `seq/mandelbrot_perf.cpp`: timers em torno de cada trecho do baseline, sem alterar a lógica de cálculo nem a entrada. Execução de referência: 1600 x 1600.

Trecho	Tempo (s)	% do total
Alocação do vetor hist	0.026051	1,42 %
Laços do Mandelbrot	1.808088	98,47 %
Somatório do checksum	0.002012	0,11 %
TOTAL	1.836151	100,00 %

Hotspot: o laço duplo sobre y e x , e dentro dele o laço while de iteração complexa. É onde está 98,47 % do tempo. A alocação e o somatório são lineares em $W.H$ e sem trabalho aritmético relevante. Vale observar que a alocação não é tempo de CPU e sim de banda de memória: `vector<long> hist(W*H)` zera 20 MB, e por isso essa parcela não encolhe com mais núcleos.

3.1 Evidência do desbalanceamento de carga

O mesmo executável acumula o total de iterações por faixa de 160 linhas. A soma das dez faixas reproduz exatamente o checksum (454.328.600), o que valida a contagem.

Faixa de linhas	Iterações	Faixa de linhas	Iterações
0-159	671.344	800-959	113.025.968
160-319	5.631.679	960-1119	73.973.767
320-479	33.987.192	1120-1279	34.278.457
480-639	73.793.246	1280-1439	5.726.769

640-799	112.567.023	1440-1599	673.155
---------	-------------	-----------	---------

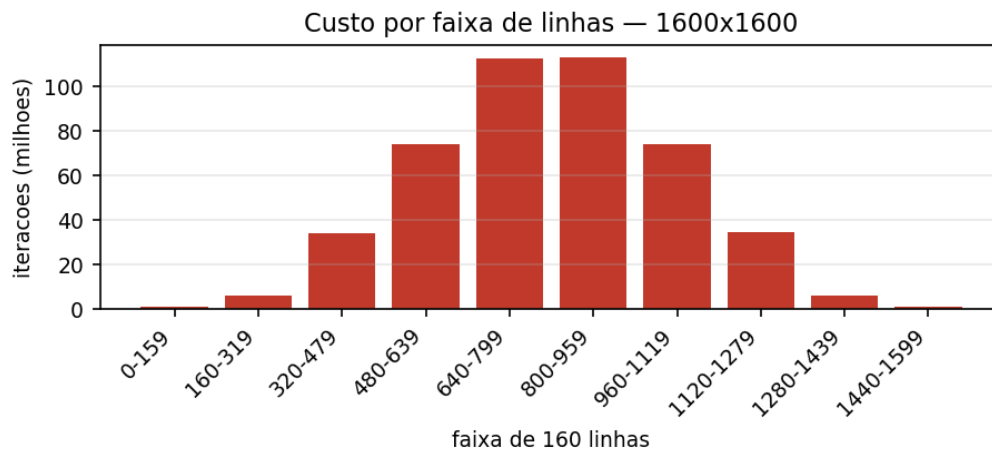


Figura 1 - A faixa mais cara custa 168 vezes mais que a mais barata.

Esse é o fato central do problema: o trabalho não é uniforme no domínio. Qualquer divisão em blocos contíguos iguais entrega quantidades de trabalho muito diferentes a cada unidade de processamento. Note também a simetria: as faixas 0-799 somam 226,7 milhões de iterações e as faixas 800-1599 somam 227,7 milhões. Essa simetria terá consequência direta no comportamento da decomposição em blocos, discutida na Seção 6.

4. Baseline medido

Protocolo de medicao. Cada configuração é executada quatro vezes: a primeira é descartada (aquecimento de cache e de páginas de memória) e as três restantes tem a mediana tomada como valor reportado. A mediana é preferida a média por ser insensível a uma execução isolada perturbada por outro processo do sistema. A entrada do baseline é determinística, com semente fixa, o que torna as execuções diretamente comparáveis. O protocolo está automatizado em scripts/medir.py, que ao final confere se todos os checksums batem com o do baseline.

Entrada (W x H)	Tempo mediano (s)	Checksum	Tempo / pixel (ns)
600 x 600	0.256884	63.906.087	713.6
1000 x 1000	0.726970	177.507.041	727.0
1600 x 1600	1.706841	454.328.600	666.7
2200 x 2200	3.248991	858.878.868	671.3

O tempo por pixel é praticamente constante entre 600 e 2200, o que confirma o crescimento proporcional a W.H previsto na análise assintótica e indica que o programa não está limitado por cache nessas faixas de tamanho.

5. Fração paralelizavel e Lei de Amdahl

A parte inerentemente sequencial é a alocação e inicialização do vetor hist. Os laços do Mandelbrot são totalmente independentes entre pixels, sem dependência de dados entre iterações, e o somatório do checksum é uma redução, paralelizavel em $O(\log P)$ passos. Da tabela da Seção 3:

$$p = (t_{\text{lacos}} + t_{\text{somatorio}}) / t_{\text{total}} = 0.985812$$

$$S_{\text{max}} = 1 / (1 - p) = 70.48$$

P	2	4	6	8	12	infinito
S(P) previsto	1.97	3.84	5.60	7.28	10.38	70.5

Observação crítica. A fração serial medida é muito pequena, de modo que Amdahl preve speedup quase linear. Isso significa que o afastamento que será observado no Marco 2 não vira da fração serial. As causas esperadas são outras: desbalanceamento de carga, overhead de escalonamento, custo de comunicação em MPI e o fato de a máquina ter 6 núcleos físicos e não 12. Amdahl aqui funciona como teto teórico otimista, não como explicação do resultado.

6. Plano de paralelização

6.1 OpenMP (memória compartilhada)

Padrão: laço paralelo sobre as linhas (y), com `#pragma omp parallel for`. A granularidade de uma linha inteira já amortiza o custo de despacho e evita o overhead de paralelizar o laço interno.

Condições de corrida: `cr`, `ci`, `zr`, `zi` e `it` são declaradas dentro do corpo do laço, o que as torna automaticamente privadas a cada thread. Se fossem declaradas fora, seriam compartilhadas e o resultado seria não determinístico. Já a escrita em `hist[y.W + x]` usa um índice distinto por pixel, então não há corrida e nenhum mecanismo de exclusão é necessário; como cada linha ocupa $W \cdot 8$ bytes, o compartilhamento falso de linha de cache também é desprezível.

Checksum: esse sim é uma redução sobre variável compartilhada e exige tratamento. Serão comparados `reduction(+:soma)`, `atomic` e `critical`, com o tempo do somatório medido separadamente, para justificar a escolha com evidência e não por convenção.

Escalonamento: dado o desbalanceamento da Seção 3.1, serão comparadas `static`, `dynamic` e `guided` com `chunks 1, 10` e padrão.

6.2 MPI (memória distribuída)

Decomposição: de domínio, por linhas da imagem. Nenhum processo precisa de dados dos vizinhos, porque o Mandelbrot não tem dependência espacial, diferentemente de um stencil. Não há troca de halo.

Duas variantes, deliberadamente: blocos contíguos e distribuição cíclica, em que o processo `r` fica com as linhas `r`, `r+P`, `r+2P` e assim por diante. A variante em blocos serve para medir o desbalanceamento; a cíclica para mostrar que a intercalação distribui linhas caras e baratas de forma quase uniforme, sem custo adicional de comunicação.

Coletivas: `MPI_Gatherv` reúne no processo 0 as linhas calculadas por todos; a variante `v` é necessária porque os pedaços têm tamanhos diferentes quando `H` não é múltiplo de `P`. `MPI_Reduce` com `MPI_MAX` e `MPI_MIN` coleta os tempos de cálculo de cada processo, o que quantifica o desbalanceamento diretamente.

Overhead de comunicação: medido isoladamente em torno do `Gatherv`. O volume transferido é fixo ($W \cdot H \cdot 8$ bytes) e independe de `P`, o que deve torná-lo uma fração crescente do tempo conforme o cálculo encolhe.

6.3 Hipóteses a verificar no Marco 2

1. Políticas que entregam linhas vizinhas a uma mesma unidade sofrerão com o desbalanceamento, e as que intercalam linhas não.
2. A decomposição cíclica em MPI aproximará o desbalanceamento de zero, enquanto a decomposição em blocos o manterá alto.
3. A eficiência cairá ao passar dos 6 núcleos físicos para as 12 threads lógicas.
4. O speedup real ficará abaixo do previsto por Amdahl, e a diferença será explicada por desbalanceamento, overhead e número de núcleos físicos, não pela fração serial.